

Fundación Kinal

Tecnología

Prof. Alejandro Max

Guia Tecnica

Despliegue de Aplicaciones Web y Buenas Prácticas de Interfaz de Usuario (UI)

Nombre: Eros Giancarlo Duarte Sánchez

Código Academico: PE5EM

Código Técnica: IN5BM

Carnet: 2025239

1. Proceso de Despliegue de una Aplicación Web

El despliegue (o deployment) es el proceso de trasladar una aplicación desde el entorno de desarrollo hacia un servidor real accesible por los usuarios finales. No se trata únicamente de copiar archivos: implica configurar el servidor, gestionar dependencias, conectar la base de datos y verificar que todo funcione correctamente antes de abrir el acceso al público.

1.1 Preparación del Entorno

Antes de transferir cualquier archivo al servidor, es necesario verificar que cuente con todos los recursos y configuraciones requeridos por la aplicación. Los aspectos principales a revisar son:

- Sistema operativo: La mayoría de servidores de producción utilizan Linux, específicamente Ubuntu Server 22.04 LTS, por su estabilidad y soporte a largo plazo.
- Runtime de la aplicación: Dependiendo del lenguaje, se instala el entorno de ejecución correspondiente: JDK 17 o superior para Java, Node.js 20 LTS para aplicaciones JavaScript.
- Servidor web: Se configura Nginx o Apache HTTP Server para gestionar solicitudes entrantes, o se utiliza el servidor embebido de frameworks como Spring Boot.
- Acceso SSH con permisos sudo para instalar paquetes y gestionar servicios del sistema.
- Puertos en el firewall: Puerto 80 para HTTP y puerto 443 para HTTPS deben estar habilitados.

1.2 Gestión de Dependencias

Las aplicaciones modernas dependen de bibliotecas externas que deben estar disponibles en el servidor con las versiones correctas. Las herramientas varían según el lenguaje:

- Java con Maven: Se ejecuta `mvn dependency:resolve` para descargar y verificar todas las dependencias definidas en `pom.xml`.
- Node.js: Se utiliza `npm ci` en lugar de `npm install`, ya que `npm ci` instala exactamente lo indicado en `package-lock.json`, garantizando reproducibilidad entre entornos.
- Python: Se crea un entorno virtual con `python -m venv venv` y se instalan las dependencias con `pip install -r requirements.txt`.

El archivo que lista las dependencias siempre debe estar guardado y actualizado en el repositorio de código.

1.3 Creación del Paquete de Distribución

El paquete de distribución es el artefacto final listo para desplegarse. Su formato depende de la tecnología empleada:

- Spring Boot: El comando `mvn clean package -DskipTests` genera un archivo `.jar` autoejecutable que incluye el servidor web embebido.
- React / Node.js: El comando `npm run build` genera la carpeta `dist/` con archivos estáticos optimizados y minificados para producción.

- Docker: El comando `docker build -t nombre-app:version` crea una imagen que empaqueta código, runtime y dependencias en una unidad portable e inmutable.

Antes de continuar, se recomienda ejecutar pruebas básicas sobre el paquete generado en un entorno aislado para detectar posibles errores antes de llegar a producción.

1.4 Configuración del Servidor

- Variables de entorno: Las cadenas de conexión, claves de API y datos sensibles no deben escribirse en el código. Se definen como variables de entorno o se gestionan con herramientas como HashiCorp Vault o AWS Secrets Manager.

- Proxy inverso: Nginx se configura para recibir el tráfico externo y redirigirlo a la aplicación, además de gestionar la carga de archivos estáticos como imágenes y hojas de estilo.

- HTTPS: Obligatorio en producción. Se obtiene un certificado SSL/TLS gratuito con Let's Encrypt mediante la herramienta Certbot.

- Gestor de procesos: Para reinicio automático ante fallos se usa `systemd` en Linux, `PM2` para Node.js, o Docker como orquestador de contenedores.

1.5 Conexión con la Base de Datos

La conectividad con la base de datos debe verificarse antes de iniciar la aplicación en producción. Se comprueba que los parámetros de conexión (host, puerto, nombre, usuario y contraseña) correspondan al entorno de producción. Los cambios de esquema se aplican con Flyway o Liquibase antes de iniciar. Se configura un pool de conexiones — HikariCP en Java con Spring Boot— y se realiza una prueba básica de conectividad (`SELECT 1`) para confirmar que la base de datos responde correctamente.

1.6 Transferencia, Arranque y Verificación

El artefacto se copia al servidor con SCP (`scp target/app.jar usuario@servidor:/opt/app/`), se detiene la versión anterior con `SIGTERM` para un apagado limpio, y se inicia la nueva versión con el perfil de producción: `java -jar app.jar --spring.profiles.active=production`. Se revisan los logs en tiempo real con `tail -f`.

Una vez en ejecución, se valida el despliegue con pruebas de humo: los endpoints principales deben responder HTTP 200. Se analizan los logs buscando errores o excepciones, y se monitorea el uso de CPU y memoria con herramientas como Prometheus y Grafana. Se configuran alertas automáticas para notificar al equipo ante caídas o degradación del rendimiento.

2. Entornos de Trabajo

En el ciclo de vida del software se distinguen tres entornos principales que deben mantenerse separados para evitar errores que afecten a los usuarios reales.

Desarrollo: Entorno local del programador con herramientas de depuración activas, datos ficticios y configuración permisiva. Los errores son parte normal del proceso y no afectan a usuarios finales. Archivo de configuración: `.env.development`.

Staging (Pruebas / QA): Replica las condiciones de producción lo más fielmente posible. Toda actualización se valida aquí con datos anonimizados o sintéticos antes de publicarse. Solo accede el equipo de QA y el cliente. Las herramientas de debug están desactivadas; únicamente se permite logging. Archivo: `.env.staging`.

Producción: Entorno disponible para todos los usuarios finales. Cualquier error tiene impacto directo: usuarios sin acceso, posible pérdida de datos o tiempo de inactividad. Todos los cambios deben pasar por los entornos anteriores, estar documentados y aprobados. Las herramientas de debug están completamente desactivadas. Archivo: `.env.production` (protegido con datos sensibles).

Diferencias clave — Objetivo: escribir código · validar · servir usuarios. Datos: ficticios · anonimizados · reales. Acceso: equipo dev · QA y cliente · público general. Estabilidad: baja (cambios frecuentes) · media (ciclos de release) · alta (disponibilidad continua).

3. Herramientas y Servicios para el Despliegue

A continuación se describen las principales herramientas que facilitan, automatizan y hacen más seguro el proceso de despliegue en la industria del desarrollo de software.

Spring Boot: Framework Java con servidor web embebido (Tomcat o Undertow); no requiere instalación de servidor separado. El módulo Spring Actuator expone los endpoints `/health` y `/metrics` para monitoreo del estado de la aplicación. Comando de producción: `java -jar app.jar --spring.profiles.active=prod`.

Apache Tomcat: Servidor de aplicaciones Java independiente con amplia trayectoria en la industria. Las aplicaciones se despliegan en formato `.war` dentro de la carpeta `webapps/`. Soporta múltiples aplicaciones en el mismo servidor y cuenta con una interfaz web de administración (Manager App) para redespigie sin reiniciar el servidor.

Docker: Empaqueta código, runtime y dependencias en una imagen portable que se comporta igual en cualquier servidor, eliminando inconsistencias entre entornos. Docker Compose permite orquestar múltiples contenedores (aplicación, base de datos, caché) con un único archivo `docker-compose.yml`.

Plataformas en la nube: AWS, Google Cloud y Azure ofrecen infraestructura escalable para proyectos de cualquier tamaño. Heroku, Railway y Render simplifican el proceso conectando un repositorio de GitHub y desplegando automáticamente con cada nuevo commit. Todas integran nativamente pipelines de CI/CD.

CI/CD (Integración y Despliegue Continuo): Automatiza las etapas de compilación, prueba y despliegue. Un pipeline típico con GitHub Actions incluye: detección del push a la rama main → compilación del código → pruebas unitarias y de integración → análisis estático con SonarQube → despliegue a staging → despliegue a producción (automático o con aprobación manual).

4. Buenas Prácticas de Interfaz de Usuario (UI)

Una aplicación puede estar correctamente programada, pero si la interfaz es confusa, los usuarios la abandonarán. El diseño de interfaz tiene como objetivo que cualquier persona pueda usar la aplicación de forma intuitiva, eficiente y sin frustraciones, independientemente de sus habilidades técnicas.

4.1 Claridad Visual

Reduce la carga cognitiva del usuario. El uso estratégico del espacio en blanco, la alineación consistente de elementos y la eliminación de contenido redundante permiten identificar rápidamente la información y las acciones disponibles. La Ley de Prägnanz establece que el cerebro interpreta los estímulos de la forma más simple posible; un buen diseño aprovecha esto agrupando elementos relacionados y eliminando todo lo que no aporte valor.

4.2 Consistencia de Diseño

Genera familiaridad y reduce errores. Cuando los elementos se comportan siempre de la misma forma, el usuario aprende la interfaz más rápido. Abarca la dimensión visual (colores, tipografías e íconos uniformes) y la funcional (mismas interacciones → mismos resultados en toda la app). Se recomienda definir un sistema de diseño documentado; Material Design de Google y Fluent Design de Microsoft son referencias de la industria.

4.3 Accesibilidad (a11y)

Amplía el alcance a usuarios con discapacidades visuales, motoras, auditivas o cognitivas, mejorando la experiencia para todos. Las pautas WCAG 2.1 definen tres niveles de cumplimiento; el nivel AA es el mínimo recomendado para aplicaciones profesionales. Requisitos clave: contraste mínimo 4.5:1 entre texto y fondo, texto alternativo descriptivo en imágenes, y navegación completa con teclado sin necesidad de mouse.

4.4 Jerarquía de la Información

Organiza el contenido de mayor a menor importancia, guiando la atención del usuario de forma natural y facilitando la comprensión. Se construye mediante el tamaño tipográfico (títulos > subtítulos > cuerpo), el peso de fuente (negrita para elementos clave), la saturación del color (más intenso = más prioritario) y la posición en pantalla.

4.5 Uso Adecuado del Color

Los colores comunican significado. Se recomienda una paleta de máximo tres colores con intención semántica clara: uno primario para acciones principales, uno secundario para soporte y uno de acento para alertas o estados especiales. Convención estándar: verde → éxito, rojo → error, amarillo → advertencia. Nunca se debe transmitir información exclusivamente por color, ya que los usuarios con daltonismo no podrán percibirla.

4.6 Tipografía Legible

Una fuente adecuada con el tamaño correcto reduce la fatiga visual. Para interfaces digitales se prefieren fuentes sans-serif como Inter, Roboto u Open Sans, ya que ofrecen mejor legibilidad en pantalla que las fuentes con serifa (optimizadas para medios impresos). El tamaño mínimo recomendado para texto de cuerpo es 16 px con una altura de línea de 1.5.

4.7 Facilidad de Navegación

La navegación debe ser predecible y consistente: el usuario no debería pensar cómo moverse por la aplicación. Se recomienda que cualquier contenido sea accesible en máximo tres interacciones desde la pantalla principal. Los elementos más efectivos son: menú principal siempre visible, breadcrumbs para indicar la ubicación jerárquica y campo de búsqueda en aplicaciones con gran volumen de contenido.

4.8 Retroalimentación Visual

El usuario necesita confirmar que su acción tuvo efecto. Sin retroalimentación, no puede distinguir si el sistema está procesando algo o si dejó de responder. Ejemplos: botón que muestra 'Guardando...' con indicador de carga mientras se procesa, notificación de éxito o error al finalizar la operación. Esta práctica aumenta la confianza en el sistema.

4.9 Diseño Responsivo

Garantiza una experiencia consistente en dispositivos de cualquier tamaño. Dado que la mayoría del tráfico web proviene de dispositivos móviles, es imprescindible implementar grids fluidos, imágenes que escalan según el viewport y elementos táctiles de al menos 48x48 px para facilitar la interacción con pantallas táctiles.

4.10 Claridad en el Call To Action (CTA)

Un Call To Action claro guía al usuario hacia el objetivo del flujo, aumentando las conversiones y reduciendo el abandono de la interfaz. Se recomienda un único botón primario por pantalla, con texto descriptivo y específico como 'Crear cuenta' o 'Guardar cambios', en lugar de textos genéricos como 'Aceptar' u 'OK'.

5. Buenas Prácticas Generales de Despliegue

Independientemente de la tecnología utilizada, las siguientes prácticas transversales garantizan despliegues seguros, ordenados y fáciles de revertir ante problemas.

5.1 Control de Versiones y Seguridad

- Todo el código fuente, scripts de infraestructura y archivos CI/CD deben estar en Git con feature branching y revisiones mediante Pull Requests.
- Cada versión desplegada a producción se etiqueta con versionado semántico: MAYOR.MENOR.PARCHE (ej. v2.1.0).

- Las contraseñas, tokens y claves de API nunca deben incluirse en el código fuente ni en el repositorio, aunque sea privado.
- HTTPS debe habilitarse en todos los entornos. Las dependencias se analizan periódicamente con OWASP Dependency-Check o Snyk dentro del pipeline CI/CD.

5.2 Estrategias de Despliegue sin Interrupción

Cuando la aplicación ya tiene usuarios activos, actualizar la versión sin generar tiempo de inactividad es un requisito crítico. Las estrategias más utilizadas son:

Blue-Green Deployment: Se mantienen dos entornos de producción idénticos. Se actualiza el entorno inactivo y luego se redirige el tráfico hacia él. Ante problemas, el cambio se revierte en segundos.

Rolling Update: Las instancias se actualizan gradualmente, manteniendo siempre un porcentaje con la versión anterior activa mientras el resto se actualiza.

Canary Release: Solo el 5–10% del tráfico se dirige a la nueva versión. Si no se detectan incidentes, el porcentaje se incrementa progresivamente hasta el 100%.

5.3 Documentación

Mantener un CHANGELOG actualizado con los cambios de cada versión, un Runbook con los procedimientos de despliegue accesible a todo el equipo, y registrar un post-mortem después de cada incidente en producción documentando la causa raíz, el impacto y las acciones correctivas tomadas para evitar que se repita.